many sub-routines as possible into helper methods, that can later be implemented according to the context.

In this case, there is three of them: we'll discuss in the next section both the function computing how the temperature evolves and the one giving the probability of acceptance of a transition; let's talk right away, instead, about function `randomStep`, which allows us to build the next tentative solution to which the algorithm could transition.

This function needs, obviously, to be domain-dependent: the size of the problem space and the type of solutions will determine how we can change current solution (a point in the problem space). For the graph embedding problem, for instance, we can randomly move each vertex along both axes, within the maximum area in which the graph must be embedded.

But, as you can see at line #4, we added a dependency on the temperature for this function!

We mentioned, in the previous section, that you can adjust the length of the random step "directly or indirectly": the short answer is that you want to be careful if you do it directly, but that's an option; to see why, we first have to better explain in details the other two functions and understand why simulated annealing work.

Listing 17.1 A generic implementation of Simulated Annealing

```
function simulatedAnnealing(C, P0, T0, acceptance, maxSteps)
         #1
```